

Testy Funkcjonalne

SeismicDisasterDWH – Hurtownia danych sejsmicznych i katastrof naturalnych

14 czerwca 2026

1 Testy funkcjonalne

T-ETL-01: Ekstrakcja danych USGS (inicjalizacja)

Warstwa ETL
Cel Sprawdzenie, że `extract_usgs.py` pobiera dane z USGS API do `STG.USGS_Raw` przy pierwszym uruchomieniu.
Kroki
1. Uruchom `python etl/python/extract_usgs.py`.
2. `SELECT COUNT(*) FROM STG.USGS_Raw`.
3. `SELECT * FROM ETL_RunLog ORDER BY RunId DESC`.
Oczekiwany wynik / Potwierdzenie `STG.USGS_Raw > 0` wierszy, status `SUCCESS` w `ETL_RunLog`, `RowsRejected=0`.
Potwierdzenie: log konsoli + zrzut `ETL_RunLog`.

```
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> py -3.10 etl\python\extract_usgs.py
2026-06-14 17:06:05,855 [INFO] Fetching USGS data for 2026 ...
2026-06-14 17:06:12,844 [INFO] Year 2026: 0 loaded, 0 rejected.
2026-06-14 17:06:12,860 [INFO] Done. Total loaded: 0, rejected: 0.
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterSTG -Q "
>> SELECT COUNT(*) AS STG_USGS_Raw_count FROM STG_USGS_Raw;
>> "
STG_USGS_Raw_count
-----
184718
(1 rows affected)
```

Log konsoli z uruchomienia `extract_usgs.py` – ekstrakcja zakończona sukcesem.

```
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterSTG -Q "
>> SELECT * FROM ETL_RunLog ORDER BY RunId DESC
>> "
RunId      SourceSystem  RunStart      RunEnd      Status      RowsExtracted RowsLoaded  RowsRejected ErrorMessage      ExtractFrom      ExtractTo
-----
17 USGS      2026-06-14 17:06:05.857 2026-06-14 17:06:12.860 SUCCESS      NULL          0          0 NULL              2026-01-01      2026-06-14
16 USGS      2026-06-14 17:04:37.390 2026-06-14 17:04:45.697 SUCCESS      NULL          3          0 NULL              2026-01-01      2026-06-14
15 USGS      2026-06-13 23:37:47.790 2026-06-13 23:43:24.590 SUCCESS      NULL      184715          0 NULL              2026-01-01      2026-06-13
12 EMDAT      2026-06-13 20:14:51.250 2026-06-13 20:14:51.250 SUCCESS      NULL          0          NULL NULL              NULL            NULL
11 EMDAT      2026-06-13 20:14:35.803 2026-06-13 20:14:35.803 SUCCESS      NULL          0          NULL NULL              NULL            NULL
10 EMDAT      2026-06-13 19:56:46.680 2026-06-13 19:56:46.800 SUCCESS      NULL          698          NULL NULL              NULL            NULL
9 EMDAT      2026-06-13 19:25:19.143 2026-06-13 19:25:20.110 SUCCESS      NULL          698          NULL NULL              NULL            NULL
(7 rows affected)
```

Wpis w `ETL_RunLog` ze statusem `SUCCESS`, `RowsRejected=0`.

T-ETL-02: Ekstrakcja przyrostowa (kolejna iteracja)

Warstwa ETL – scenariusz przyrostowy
Cel Sprawdzenie, że kolejne uruchomienie ekstrakcji dociąga tylko nowe zdarzenia, bez duplikacji.
Kroki
1. Zanotuj `COUNT(*)` z `STG.USGS_Raw`.
2. Uruchom ponownie `extract_usgs.py`.
3. `SELECT EventId, COUNT(*) FROM STG_USGS_Raw GROUP BY EventId HAVING COUNT(*)>1`.
Oczekiwany wynik / Potwierdzenie Nowy wpis w `ETL_RunLog`; zapytanie o duplikaty zwraca 0 wierszy.
Potwierdzenie: log konsoli + zrzut zapytania o duplikaty (0 wierszy).

```

PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> py -3.10 etl\python\extract_usgs.py
2026-06-14 17:11:18,549 [INFO] Fetching USGS data for 2026 ...
2026-06-14 17:11:25,175 [INFO] Year 2026: 0 loaded, 0 rejected.
2026-06-14 17:11:25,191 [INFO] Done. Total loaded: 0, rejected: 0.
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterSTG -Q "SELECT COUNT(*) FROM STG_USGS_Raw"

-----
184718

(1 rows affected)
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> py -3.10 etl\python\extract_usgs.py
2026-06-14 17:12:41,017 [INFO] Fetching USGS data for 2026 ...
2026-06-14 17:12:47,695 [INFO] Year 2026: 0 loaded, 0 rejected.
2026-06-14 17:12:47,695 [INFO] Done. Total loaded: 0, rejected: 0.
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterSTG -Q "
>> SELECT EventId, COUNT(*) FROM STG_USGS_Raw GROUP BY EventId HAVING COUNT(*)>1;
>> SELECT COUNT(*) FROM STG_USGS_Raw;
>> SELECT TOP 3 RunId, SourceSystem, Status, RowsLoaded, RowsRejected, ExtractFrom, ExtractTo FROM ETL_RunLog ORDER BY RunId DESC;
>> "
EventId
-----
(0 rows affected)

-----
184718

(1 rows affected)
RunId      SourceSystem      Status      RowsLoaded  RowsRejected  ExtractFrom      ExtractTo
-----
19 USGS           SUCCESS      0           0           2026-01-01      2026-06-14
18 USGS           SUCCESS      0           0           2026-01-01      2026-06-14
17 USGS           SUCCESS      0           0           2026-01-01      2026-06-14

(3 rows affected)

```

Kolejne uruchomienia ekstrakcji (RowsLoaded=0), STG_USGS_Raw bez zmian, 0 duplikatów EventId.

T-DWH-01: Schemat hurtowni danych

Warstwa	DWH – struktura
Cel	Sprawdzenie, że skrypt tworzy baze SeismicDisasterDWH z kompletnym zestawem tabel.
Kroki	1. sqlcmd -i Skrypt_SQL_Hurtownia_tsunami.sql. 2. SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES. 3. EXEC sp_help 'dbo.FactSeismic'.
Oczekiwany wynik / Potwierdzenie	Wszystkie 9 tabel (DimDate, DimGeography, DimMagnitude, DimSeismicDepth, DimSeverityDeaths, DimSeverityAffected, FactSeismic, FactDisaster, BridgeDisasterSeismic) istnieją; typy kolumn zgodne z konwencją. <i>Potwierdzenie:</i> zrzut listy tabel + zrzut sp_help.

```

PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q
"SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE='BASE TABLE' ORDER BY TABLE_NAME"
TABLE_NAME
-----
BridgeDisasterSeismic
DimDate
DimGeography
DimMagnitude
DimSeismicDepth
DimSeverityAffected
DimSeverityDeaths
ETL_FactDisasterLoad
ETL_FactSeismicLoad
FactDisaster
FactSeismic
REF_CountryMaster

(12 rows affected)

```

Lista tabel w bazie SeismicDisasterDWH – wszystkie 9 wymaganych tabel obecne.

Struktura kolumn tabeli **FactSeismic** – typy zgodne z konwencja projektu.

T-DWH-07: Transformacja – mostek Disaster–Seismic (Haversine)

Warstwa

DWH – transformacja

Cel

Sprawdzenie, że usp_Build_BridgeDisasterSeismic łączy katastrofy ze zdarzeniami sejsmicznymi i poprawnie liczy DistanceKM/TimeLagDays.

Kroki

1. EXEC usp_Build_BridgeDisasterSeismic.
2. SELECT MIN(DistanceKM), MAX(DistanceKM), MIN(TimeLagDays), MAX(TimeLagDays) FROM BridgeDisasterSeismic.
3. Recznie przelicz Haversine dla jednej pary.

Oczekiwany wynik /

Wartości nieujemne, w rozsądnym zakresie; wynik proc. zgodny z ręcznym przeliczeniem (tolerancja ± 1 km).

Potwierdzenie

Potwierdzenie: zrzut statystyk MIN/MAX + porównanie ręcznego przeliczenia.

```
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "EXEC usp_Build_BridgeDisasterSeismic" -w 200
BridgeDisasterSeismic: 0 links inserted.
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT MIN(DistanceKM), MAX(Di
stanceKM), MIN(TimeLagDays), MAX(TimeLagDays), COUNT(*) FROM BridgeDisasterSeismic" -w 200

-----
.000 99.897 0 3 4291
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT TOP 1 b.DisasterKey, b.
SeismicKey, b.DistanceKM, b.TimeLagDays, s.Latitude AS SeisLat, s.Longitude AS SeisLon, f.Latitude AS DisLat, f.Longitude AS DisLon FROM BridgeDisas
terSeismic b JOIN FactSeismic s ON s.SeismicKey=b.SeismicKey JOIN FactDisaster f ON f.DisasterKey=b.DisasterKey WHERE b.DistanceKM > 10 ORDER BY b.D
isasterKey" -w 200
DisasterKey SeismicKey DistanceKM TimeLagDays SeisLat SeisLon DisLat DisLon
-----
2097 417010 10.283 0 25.59100 101.16400 25.60700 101.06300
```

Statystyki BridgeDisasterSeismic (DistanceKM 0–99.897, TimeLagDays 0–3, 4291 par) oraz ręczne przeliczenie Haversine dla wybranej pary (10.28 km vs 10.283 km z procedury, różnica < 1 km).

T-QA-01: Spójność – integralność FK (orphan records)

Warstwa

Spójność danych – DWH

Cel

Sprawdzenie, że każdy klucz obcy w FactSeismic/FactDisaster wskazuje na istniejący rekord wymiaru.

Kroki

Dla każdego FK: SELECT COUNT(*) FROM Fact f LEFT JOIN Dim d ON f.Key=d.Key WHERE d.Key IS NULL.

Oczekiwany wynik /

Wszystkie zapytania zwracają 0 – brak rekordów-sierot.

Potwierdzenie

Potwierdzenie: zrzut wyników dla wszystkich FK (lista + wynik 0).

```
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactSeismic.DateKey' AS FK, COUNT(*) AS Orphans FR
OM FactSeismic f LEFT JOIN DimDate d ON f.DateKey=d.DateKey WHERE d.DateKey IS NULL" -w 200
FK
-----
FactSeismic.DateKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactSeismic.GeographyKey' AS FK, COUNT(*) AS Orpha
ns FROM FactSeismic f LEFT JOIN DimGeography d ON f.GeographyKey=d.GeographyKey WHERE d.GeographyKey IS NULL" -w 200
FK
-----
FactSeismic.GeographyKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactSeismic.MagnitudeKey' AS FK, COUNT(*) AS Orpha
ns FROM FactSeismic f LEFT JOIN DimMagnitude d ON f.MagnitudeKey=d.MagnitudeKey WHERE d.MagnitudeKey IS NULL" -w 200
FK
-----
FactSeismic.MagnitudeKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactSeismic.SeismicDepthKey' AS FK, COUNT(*) AS Or
phans FROM FactSeismic f LEFT JOIN DimSeismicDepth d ON f.SeismicDepthKey=d.SeismicDepthKey WHERE d.SeismicDepthKey IS NULL" -w 200
FK
-----
FactSeismic.SeismicDepthKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactDisaster.StartDateKey' AS FK, COUNT(*) AS Orph
ans FROM FactDisaster f LEFT JOIN DimDate d ON f.StartDate=d.DateKey WHERE d.DateKey IS NULL" -w 200
FK
-----
FactDisaster.StartDateKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactDisaster.EndDateKey' AS FK, COUNT(*) AS Orphan
s FROM FactDisaster f LEFT JOIN DimDate d ON f.EndDate=d.DateKey WHERE d.DateKey IS NULL" -w 200
FK
-----
FactDisaster.EndDateKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactDisaster.GeographyKey' AS FK, COUNT(*) AS Orh
phans FROM FactDisaster f LEFT JOIN DimGeography d ON f.GeographyKey=d.GeographyKey WHERE d.GeographyKey IS NULL" -w 200
FK
-----
FactDisaster.GeographyKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactDisaster.SeverityDeathsKey' AS FK, COUNT(*) AS
Orphans FROM FactDisaster f LEFT JOIN DimSeverityDeaths d ON f.SeverityDeathsKey=d.SeverityDeathsKey WHERE d.SeverityDeathsKey IS NULL" -w 200
FK
-----
FactDisaster.SeverityDeathsKey
-----
0
PS C:\Users\Mateu\Github\DWH\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactDisaster.SeverityAffectedKey' AS FK, COUNT(*)
AS Orphans FROM FactDisaster f LEFT JOIN DimSeverityAffected d ON f.SeverityAffectedKey=d.SeverityAffectedKey WHERE d.SeverityAffectedKey IS NULL" -w 200
FK
-----
FactDisaster.SeverityAffectedKey
-----
0
```

Wyniki 9 zapytań sprawdzających klucze obce w FactSeismic i FactDisaster – wszystkie zwracają Orphans=0.

T-QA-02: Spójność – rekonyliacja liczności staging vs. fakt

Warstwa

Spójność danych – cross-layer

Cel

Sprawdzenie, że liczba rekordów nie gubi się między STG_USGS_Raw/STG_EM DAT_Raw a FactSeismic/FactDisaster.

Kroki

1. `stg_count = COUNT(*) FROM STG_*_Raw.`
2. `fact_count = COUNT(*) FROM Fact*.`
3. `diff = stg_count - fact_count;` zidentyfikuj kategorie różnicy (rejected, no_band_match itd.).

Oczekiwany wynik / Potwierdzenie

PASS gdy `stg_count = fact_count + suma wyjaśnionych różnic.`
FAIL natychmiastowy: `fact_count > stg_count` (błąd idempotentności).
Potwierdzenie: tabela rekonyliacji `src/stg/fact/diff` + status PASS/FAIL.

```
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterSTG -Q "SET NOCOUNT ON; SELECT 'STG_USGS_Raw' AS src, COUNT(*) AS stg_count FROM STG_USGS_Raw" -w 200
src          stg_count
-----
STG_USGS_Raw 184718
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactSeismic' AS fact, COUNT(*) AS fact_count FROM FactSeismic" -w 200
fact          fact_count
-----
FactSeismic  184685
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT COUNT(*) AS no_mag_band FROM SeismicDisasterSTG.dbo.STG_USGS_Raw s WHERE NOT EXISTS (SELECT 1 FROM DimMagnitude m WHERE s.Mag>=m.LowerBound AND s.Mag<=m.UpperBound)" -w 200
no_mag_band
-----
18
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT COUNT(*) AS no_depth_band FROM SeismicDisasterSTG.dbo.STG_USGS_Raw s WHERE NOT EXISTS (SELECT 1 FROM DimSeismicDepth d WHERE COALESCE(s.Depth,0)>=d.LowerBoundKM AND COALESCE(s.Depth,0)<=d.UpperBoundKM)" -w 200
no_depth_band
-----
12
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterSTG -Q "SET NOCOUNT ON; SELECT 'STG_EMDAT_Raw' AS src, COUNT(*) AS stg_count FROM STG_EMDAT_Raw" -w 200
src          stg_count
-----
STG_EMDAT_Raw 698
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT 'FactDisaster' AS fact, COUNT(*) AS fact_count FROM FactDisaster" -w 200
fact          fact_count
-----
FactDisaster  698
```

Rekoncyliacja: EMDAT 698/698 (diff=0); USGS 184718/184685 (diff=33 = 18 no_mag_band + 12 no_depth_band + 3 nieprzeładowane rekordy z T-ETL-01) – wszystkie różnice wyjaśnione, status PASS.

T-RPT-03: Raport – Case Study (mapa, pokrycie ubezpieczeniowe)

Warstwa

Raporty – Power BI

Cel

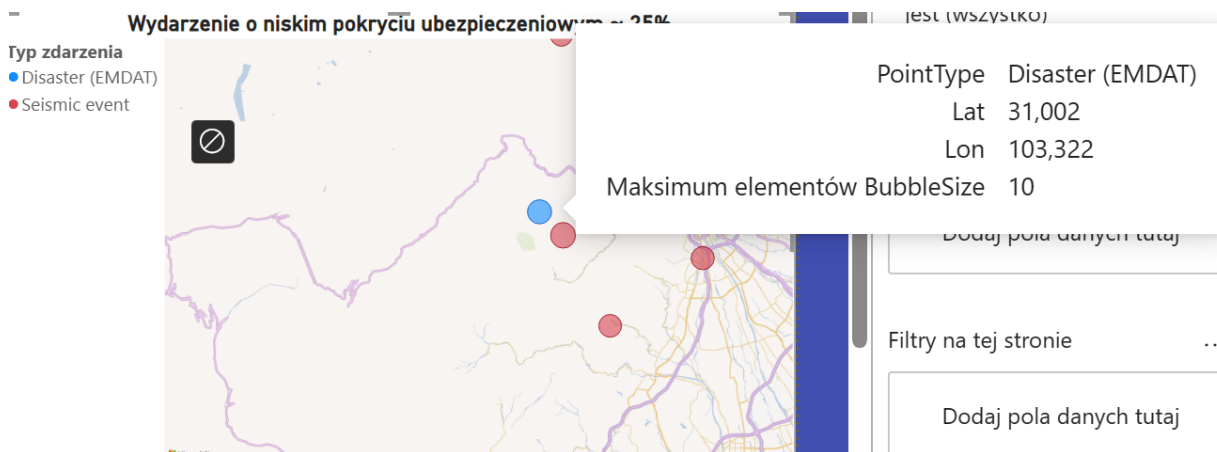
Sprawdzenie, że mapa przypadków (Christchurch DisasterKey=2403) pokazuje poprawne lokalizacje i wartości pokrycia ubezpieczeniowego z FactDisaster.

Kroki

1. Otwórz stronę "Japonia – Case Study" w raporcie Power BI.
2. Najedź na punkt Disaster (EMDAT) i odczytaj tooltip (Lat, Lon, % pokrycia).
3. Porównaj z `SELECT Latitude, Longitude, InsuredDamage/TotalDamageAdj FROM FactDisaster WHERE DisasterKey=2403.`

Oczekiwany wynik / Potwierdzenie

Lokalizacja i wartość pokrycia w tooltipie zgodne z FactDisaster (tolerancja zaokrąglenia).
Potwierdzenie: zrzuty ekranu mapy z tooltipem + zrzut zapytania SQL.

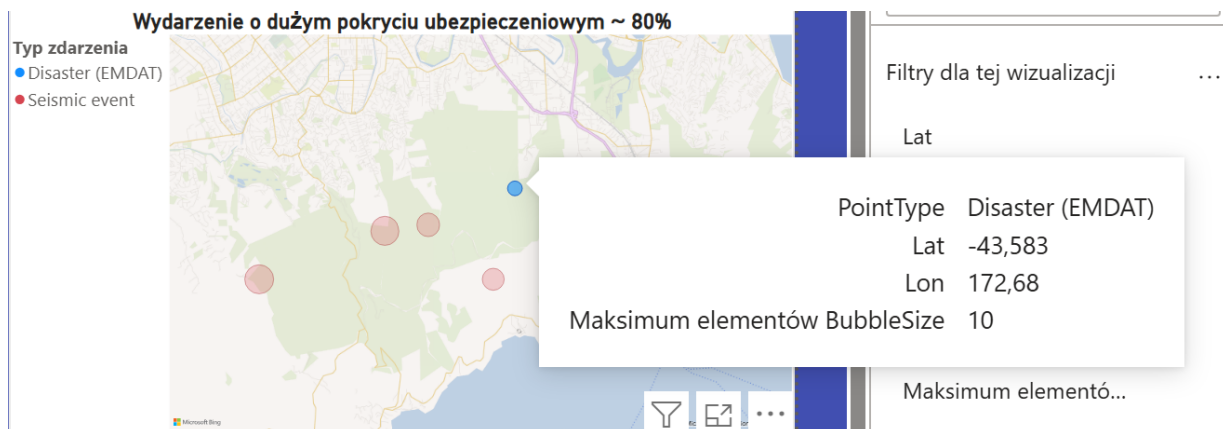


Wynik zapytania SQL dla DisasterKey=2403: Lat=-43.583, Lon=172.680, pokrycie ubezpieczeniowe $\approx 80\%$.


```
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT DisasterKey, Latitude, Longitude, TotalDamageAdj, InsuredDamage, CAST(InsuredDamage AS FLOAT)/NULLIF(TotalDamageAdj,0)*100 AS CoveragePct FROM FactDisaster WHERE DisasterKey IN (2403,2340)" -w 200
```

DisasterKey	Latitude	Longitude	TotalDamageAdj	InsuredDamage	CoveragePct
2340	31.00200	103.32200	127100952000	448592000	0.35294149488353166
2403	-43.58300	172.68000	21468668000	17174934000	79.99999813681967

Strona raportu "Japonia – Case Study- wizualizacja "Wydarzenie o dużym pokryciu ubezpieczeniowym ~80%".



Tooltip mapy: PointType=Disaster (EMDAT), Lat=-43,583, Lon=172,68 – zgodne z danymi z FactDisaster.

T-E2E-01: End-to-end – pełny przebieg pipeline'u

- Warstwa** End-to-End
- Cel** Potwierdzenie całościowego działania: od USGS/EM-DAT przez staging, DWH, do raportu Power BI dla jednego referencyjnego zdarzenia (Christchurch 2011, DisasterKey=2403).
- Kroki**
- Wykonaj kroki 1–7 z sekcji "ETL Execution Order".
 - Odśwież model/raport Power BI.
 - Dla zdarzenia referencyjnego sprawdź wartości na każdym etapie: źródło -> staging -> fakt -> mostek -> raport.
- Oczekiwany wynik / Potwierdzenie** Każdy krok bez błędów; wartości referencyjnego zdarzenia zgodne na każdym etapie.
- Potwierdzenie:* log wykonania wszystkich kroków + tabela "śledzenia" zdarzenia przez warstwy + zrzut raportu.

```
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT f.DisasterKey, f.StartDate, f.Latitude, f.Longitude, f.TotalDeaths, f.TotalDamageAdj, f.InsuredDamage, f.Tsunami, g.CountryName FROM FactDisaster f JOIN DimGeography g ON g.GeographyKey=f.GeographyKey WHERE f.DisasterKey=2403" -w 200
```

DisasterKey	StartDate	Latitude	Longitude	TotalDeaths	TotalDamageAdj	InsuredDamage	Tsunami	CountryName
2403	20110222	-43.58300	172.68000	181	21468668000	17174934000	0	New Zealand

```
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT b.SeismicKey, b.DistanceKM, b.TimeLagDays, s.DateKey, s.Magnitude, s.SeismicDepth, s.Latitude, s.Longitude FROM BridgeDisasterSeismic b JOIN FactSeismic s ON s.SeismicKey=b.SeismicKey WHERE b.DisasterKey=2403 ORDER BY b.DistanceKM" -w 200
```

SeismicKey	DistanceKM	TimeLagDays	DateKey	Magnitude	SeismicDepth	Latitude	Longitude
500319	0.000	1	20110221	6.10	5.90	-43.58300	172.68000
499881	1.716	0	20110222	4.70	5.00	-43.59800	172.67500
499879	1.744	1	20110221	4.80	5.00	-43.59000	172.66000
499746	2.539	0	20110222	5.50	5.90	-43.59000	172.65000
499575	3.464	0	20110222	4.50	4.10	-43.58300	172.72300
500066	5.036	0	20110222	5.60	6.60	-43.59000	172.62100

```
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT TOP 1 StgId, EventId, SrcTime, Mag, Depth, Latitude, Longitude, ISO3 FROM STG_USGS_Raw WHERE Latitude=-43.583 AND Longitude=172.68 AND Mag=6.1" -w 200
```

StgId	EventId	SrcTime	Mag	Depth	Latitude	Longitude	ISO3
484136	usp000huvq	2011-02-21T23:51:42.350000+00:	6.10	5.90	-43.58300	172.68000	NZL

```
PS C:\Users\Mateu\Github\DW\Geoseismic_Risk_DWH> sqlcmd -S localhost -E -C -d SeismicDisasterDWH -Q "SET NOCOUNT ON; SELECT StgId, DisNo, ISO, StartYear, StartMonth, StartDay, TotalDeaths, TotalDamageAdj, InsuredDamagesAdj FROM STG_EM-DAT_Raw WHERE ISO='NZL' AND StartYear=2011" -w 200
```

StgId	DisNo	ISO	StartYear	StartMonth	StartDay	TotalDeaths	TotalDamageAdj	InsuredDamagesAdj
998	2011-0388-NZL	NZL	2011	6	13	1	4293734	2862489
1347	2011-0068-NZL	NZL	2011	2	22	181	21468668	17174934

Śledzenie zdarzenia Christchurch (EventId=usp000huvq, DisasterKey=2403) przez warstwy: USGS staging (Mag=6.10, Lat=-43.583, Lon=172.680) → FactSeismic (SeismicKey=500319) → EMDAT staging (DisNo=2011-0068-NZL, TotalDeaths=181) → FactDisaster (DisasterKey=2403, TotalDamageAdj=21 468 668 000, InsuredDamage=17 174 934 000) → Bridge (DistanceKM=0.000, TimeLagDays=1) → raport Power BI (~80% pokrycia) – wartości zgodne na każdym etapie.

Pokrycie wymagań

Wymaganie	Przypadki testowe
Każda warstwa (ETL/DWH/raporty)	T-ETL-01, T-DWH-01, T-RPT-03
End-to-end	T-E2E-01
Inicjalizacja	T-ETL-01, T-DWH-01, T-E2E-01
Ładowanie przyrostowe	T-ETL-02, T-DWH-04
Poprawność transformacji	T-DWH-07, T-QA-02
Spójność danych na każdym etapie	T-QA-01, T-QA-02, T-E2E-01

Uwagi do realizacji

Zgodnie z aktualnym stanem środowiska, warstwa BI została zrealizowana jako model i raport **Power BI** (plik `powerbi/SeismicRiskModel.pbix`) zamiast modelu tabularnego SSAS – weryfikacja warstwy BI/raportowej jest realizowana przez T-RPT-03 i T-E2E-01.